

Paper

1 Tóm tắt paper: Neural Collaborative Filtering

1.1 Bài toán chính

Paper nghiên cứu bài toán **collaborative filtering** cho recommender systems dựa trên **implicit feedback**, ví dụ:

clicks, purchases, views, pins, ratings đã được binarize.

Với implicit feedback, dữ liệu user-item được biểu diễn bằng ma trận:

$$Y \in \mathbb{R}^{M \times N},$$

trong đó M là số user và N là số item. Mỗi phần tử:

$$y_{ui} = \begin{cases} 1, & \text{nếu interaction giữa user } u \text{ và item } i \text{ được quan sát,} \\ 0, & \text{otherwise.} \end{cases}$$

Tuy nhiên:

$$y_{ui} = 1$$

không nhất thiết nghĩa là user thật sự thích item, mà chỉ nói rằng có interaction.

Tương tự:

$$y_{ui} = 0$$

không nhất thiết nghĩa là user không thích item, vì có thể user chưa từng biết đến item đó.

Do đó, implicit feedback có hai khó khăn lớn:

- feedback là noisy signal về preference thật;
- thiếu negative feedback tự nhiên.

Mục tiêu của recommender system là học một hàm dự đoán:

$$\hat{y}_{ui} = f(u, i \mid \Theta),$$

sau đó dùng $\hat{y}_{ui}$ để xếp hạng các item chưa được user tương tác.

1.2 Vấn đề với Matrix Factorization

Matrix Factorization là phương pháp phổ biến trong collaborative filtering.

Mỗi user u có latent vector:

$$p_u \in \mathbb{R}^K,$$

mỗi item i có latent vector:

$$q_i \in \mathbb{R}^K.$$

MF dự đoán interaction bằng inner product:

$$\hat{y}_{ui} = p_u^\top q_i = \sum_{k=1}^K p_{uk} q_{ik}.$$

Paper lập luận rằng MF bị giới hạn vì inner product là một interaction function quá đơn giản:

$$f(u, i) = p_u^\top q_i.$$

Inner product giả định rằng các latent dimensions tương tác tuyến tính và độc lập. Điều này có thể không đủ để mô hình hóa quan hệ phức tạp giữa user và item.

Do đó, paper đặt câu hỏi:

Thay vì cố định interaction function là inner product, liệu có thể học interaction function bằng neural

1.3 Ý tưởng chính của Neural Collaborative Filtering

Paper đề xuất framework:

$$\text{NCF} = \text{Neural Collaborative Filtering}.$$

Ý tưởng chính là thay inner product trong MF bằng một neural architecture có khả năng học một hàm tương tác tổng quát hơn:

$$\hat{y}_{ui} = f(P^\top v_u^U, Q^\top v_i^I \mid P, Q, \Theta_f).$$

Trong đó:

- v_u^U : one-hot vector biểu diễn user u ,
- v_i^I : one-hot vector biểu diễn item i ,
- P : user embedding matrix,
- Q : item embedding matrix,
- $P^\top v_u^U$: user embedding,
- $Q^\top v_i^I$: item embedding,
- Θ_f : tham số của neural interaction function.

NCF có kiến trúc tổng quát gồm:

- input layer: one-hot user ID và item ID;
- embedding layer: ánh xạ one-hot vector sang dense latent vector;
- neural collaborative filtering layers: học interaction giữa user embedding và item embedding;
- output layer: dự đoán score $\hat{y}_{ui}$.

Hình 2 trong paper minh họa framework này: user và item one-hot vectors đi qua embedding layer, sau đó được đưa vào các neural CF layers để tạo score dự đoán.

1.4 Loss function: Log Loss / Binary Cross-Entropy

Paper tập trung vào implicit feedback, nên xem recommendation như một bài toán binary classification.

Model dự đoán:

$$\hat{y}_{ui} \in [0, 1],$$

được hiểu là xác suất item i relevant với user u .

Để đảm bảo output nằm trong $[0, 1]$, paper dùng sigmoid ở output layer:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Likelihood cho observed interactions Y và sampled negative interactions Y^- là:

$$p(Y, Y^- | P, Q, \Theta_f) = \prod_{(u,i) \in Y} \hat{y}_{ui} \prod_{(u,j) \in Y^-} (1 - \hat{y}_{uj}).$$

Lấy negative log-likelihood, ta được loss:

$$\mathcal{L} = - \sum_{(u,i) \in Y} \log \hat{y}_{ui} - \sum_{(u,j) \in Y^-} \log(1 - \hat{y}_{uj}).$$

Viết gọn trên cả positive và negative samples:

$$\mathcal{L} = - \sum_{(u,i) \in Y \cup Y^-} [y_{ui} \log \hat{y}_{ui} + (1 - y_{ui}) \log(1 - \hat{y}_{ui})].$$

Đây chính là:

Binary Cross-Entropy / Log Loss.

Paper nhấn mạnh rằng squared loss không thật sự phù hợp với implicit feedback vì target là binary:

$$y_{ui} \in \{0, 1\}.$$

Do đó, log loss có diễn giải xác suất phù hợp hơn.

1.5 Negative Sampling

Do implicit feedback chỉ có positive observations, cần tạo negative instances từ unobserved interactions.

Gọi:

$$Y^-$$

là tập negative samples lấy từ các cặp user-item chưa observed.

Paper dùng uniform negative sampling và kiểm soát tỷ lệ:

negative samples per positive sample.

Thí nghiệm cho thấy một negative sample cho mỗi positive sample thường chưa đủ. Sampling nhiều negative hơn giúp cải thiện performance, nhưng nếu sampling quá nhiều thì có thể làm performance giảm.

Trong paper, tỷ lệ tốt thường khoảng:

$$3 \sim 6$$

negative samples cho mỗi positive sample.

Điểm này rất quan trọng vì nó cho thấy loss không đứng độc lập; cách tạo negative data ảnh hưởng mạnh đến objective mà model thật sự học.

1.6 Generalized Matrix Factorization

Paper chỉ ra MF có thể được xem là trường hợp đặc biệt của NCF.

Với user embedding:

$$p_u = P^\top v_u^U,$$

và item embedding:

$$q_i = Q^\top v_i^I,$$

GMF dùng element-wise product:

$$\phi_1(p_u, q_i) = p_u \odot q_i.$$

Sau đó output là:

$$\hat{y}_{ui} = a_{\text{out}}(h^\top(p_u \odot q_i)).$$

Nếu:

$$a_{\text{out}}(x) = x$$

và:

$$h = \mathbf{1},$$

thì:

$$\hat{y}_{ui} = \sum_{k=1}^K p_{uk} q_{ik} = p_u^\top q_i,$$

tức là đúng bằng Matrix Factorization.

Trong paper, tác giả dùng sigmoid output và học vector h , tạo thành:

GMF = Generalized Matrix Factorization.

GMF tổng quát hơn MF vì mỗi latent dimension có thể có trọng số khác nhau, và output có thể phi tuyến.

1.7 Multi-Layer Perceptron cho Collaborative Filtering

Thay vì dùng element-wise product như GMF, paper đề xuất dùng MLP để học interaction function.

Đầu tiên, concatenate user embedding và item embedding:

$$z_1 = \phi_1(p_u, q_i) = \begin{bmatrix} p_u \\ q_i \end{bmatrix}.$$

Sau đó đưa qua nhiều hidden layers:

$$\phi_2(z_1) = a_2(W_2^\top z_1 + b_2),$$

$$\phi_3(z_2) = a_3(W_3^\top z_2 + b_3),$$

...

$$\phi_L(z_{L-1}) = a_L(W_L^\top z_{L-1} + b_L).$$

Output:

$$\hat{y}_{ui} = \sigma(h^\top \phi_L(z_{L-1})).$$

Paper chọn ReLU cho hidden layers:

$$\text{ReLU}(x) = \max(0, x),$$

vì ReLU ít bị saturation hơn sigmoid/tanh và phù hợp với dữ liệu sparse.

Ý nghĩa:

MLP cho phép học interaction phi tuyến giữa user embedding và item embedding.

1.8 NeuMF: Neural Matrix Factorization

Paper đề xuất mô hình chính:

$$\text{NeuMF} = \text{Neural Matrix Factorization}.$$

NeuMF kết hợp hai nhánh:

- **GMF**: học interaction tuyến tính kiểu MF;
- **MLP**: học interaction phi tuyến bằng deep neural network.

Điểm quan trọng là GMF và MLP không bắt buộc chia sẻ embedding. Paper cho phép mỗi nhánh học embedding riêng:

$$p_u^G, q_i^G$$

cho GMF, và:

$$p_u^M, q_i^M$$

cho MLP.

Nhánh GMF:

$$\phi^{GMF} = p_u^G \odot q_i^G.$$

Nhánh MLP:

$$\phi^{MLP} = a_L(W_L^\top (\cdots a_2(W_2^\top \begin{bmatrix} p_u^M \\ q_i^M \end{bmatrix} + b_2) \cdots) + b_L).$$

Sau đó concatenate hai nhánh:

$$\hat{y}_{ui} = \sigma \left(h^\top \begin{bmatrix} \phi^{GMF} \\ \phi^{MLP} \end{bmatrix} \right).$$

NeuMF kết hợp:

linearity của MF + non-linearity của MLP.

Hình 3 trong paper minh họa NeuMF: một nhánh GMF dùng element-wise product, một nhánh MLP dùng concatenation và nhiều hidden layers, rồi hai nhánh được nối lại trước output layer.

1.9 Pre-training

Do NeuMF có objective non-convex, initialization ảnh hưởng mạnh đến kết quả.

Paper đề xuất pre-training:

- train GMF riêng đến khi hội tụ;
- train MLP riêng đến khi hội tụ;
- dùng tham số của GMF và MLP để khởi tạo NeuMF.

Output layer được khởi tạo bằng cách kết hợp weights của hai model:

$$h \leftarrow \begin{bmatrix} \alpha h^{GMF} \\ (1 - \alpha) h^{MLP} \end{bmatrix}.$$

Trong thí nghiệm, paper dùng:

$$\alpha = 0.5.$$

Kết quả cho thấy pre-training thường cải thiện performance của NeuMF.

1.10 Thí nghiệm

Paper đánh giá trên hai dataset:

- **MovieLens**: 1,000,209 interactions, 3,706 items, 6,040 users, sparsity 95.53%.
- **Pinterest**: 1,500,809 interactions, 9,916 items, 55,187 users, sparsity 99.73%.

MovieLens vốn là explicit rating dataset, nhưng paper binarize thành implicit feedback:

$$y_{ui} = 1 \quad \text{nếu user đã rating item.}$$

Pinterest là implicit feedback dataset, interaction là việc user pin một image.

Paper dùng leave-one-out evaluation:

- giữ lại interaction mới nhất của mỗi user làm test;
- dùng phần còn lại để train;
- random 100 negative items chưa interacted;
- rank test item cùng 100 negative items.

Metrics:

$$\text{HR@10}$$

và:

$$\text{NDCG@10}.$$

HR@10 đo test item có xuất hiện trong top-10 hay không.

NDCG@10 tính thêm vị trí của hit:

hit ở rank cao hơn $\Rightarrow$ score cao hơn.

1.11 Baselines

Paper so sánh với:

- **ItemPop**: recommend item phổ biến nhất, không personalized.
- **ItemKNN**: item-based collaborative filtering.
- **BPR**: Matrix Factorization với pairwise ranking loss.
- **eALS**: state-of-the-art MF cho implicit feedback, tối ưu squared loss có weighting.

Các mô hình đề xuất:

GMF, MLP, NeuMF.

1.12 Kết quả chính

Paper trả lời ba câu hỏi nghiên cứu.

1.12.1 RQ1: NCF có outperform state-of-the-art không?

Kết quả cho thấy:

NeuMF

đạt performance tốt nhất trên cả hai dataset.

Trung bình, NeuMF cải thiện so với eALS và BPR khoảng:

4.5% và 4.9%.

GMF và MLP cũng có performance mạnh.

Đặc biệt, GMF thường outperform BPR dù cả hai đều học dạng MF-like model. Sự khác biệt chủ yếu nằm ở objective:

GMF dùng log loss,

trong khi:

BPR dùng pairwise ranking loss.

Điều này cho thấy log loss với negative sampling là một lựa chọn hiệu quả cho implicit recommendation.

1.12.2 RQ2: Log loss với negative sampling có hiệu quả không?

Paper quan sát rằng khi training loss giảm, HR@10 và NDCG@10 thường tăng.

Tuy nhiên, sau một số iteration, training loss vẫn có thể giảm nhưng recommendation performance bắt đầu giảm, cho thấy overfitting.

Điều này minh họa một hiện tượng quen thuộc:

Loss giảm không luôn đồng nghĩa metric recommendation tăng.

Về negative sampling, paper thấy rằng:

1

negative per positive là chưa đủ, còn tỷ lệ khoảng:

$3 \sim 6$

thường tốt hơn.

Nếu sampling ratio quá lớn, performance có thể giảm.

1.12.3 RQ3: Deep learning có hữu ích không?

Paper so sánh MLP với số hidden layers khác nhau:

MLP-0, MLP-1, MLP-2, MLP-3, MLP-4.

Kết quả cho thấy nhiều hidden layers thường cải thiện HR@10 và NDCG@10.

MLP-0, tức chỉ concatenate embedding rồi chiếu thẳng ra output, có performance rất yếu.

Điều này chứng minh rằng chỉ nối user embedding và item embedding là không đủ; cần hidden layers để học interaction giữa chúng.

1.13 Liên hệ với loss function trong recommender systems

Paper này rất quan trọng cho chủ đề loss vì nó cho thấy:

- implicit feedback có bản chất one-class và noisy;
- squared loss không phù hợp hoàn toàn với target binary;
- log loss có diễn giải xác suất rõ ràng hơn;
- negative sampling ảnh hưởng trực tiếp đến objective học được;
- loss giảm có thể không đồng nhất với HR/NDCG tăng;

- cùng một model family nhưng objective khác nhau có thể cho performance khác nhau.

Nếu BPR paper nhấn mạnh:

ranking objective

thì NCF paper nhấn mạnh:

probabilistic binary classification objective.

Hai paper đại diện cho hai hướng loss lớn trong recommender systems:

BPR $\rightarrow$ pairwise ranking loss,

NCF $\rightarrow$ pointwise log loss with negative sampling.

1.14 Kết luận chính

Paper kết luận rằng:

- Deep neural networks có thể được dùng để học user-item interaction function.
- Matrix Factorization là một trường hợp đặc biệt của NCF.
- MLP giúp học nonlinear interaction giữa user và item embeddings.
- NeuMF kết hợp ưu điểm của GMF và MLP.
- Log loss với negative sampling hiệu quả cho implicit feedback recommendation.
- Deeper neural layers có thể cải thiện recommendation performance.

1.15 Thông điệp quan trọng nhất

Trong collaborative filtering, interaction function không nhất thiết phải là inner product.

NCF học interaction function bằng neural network, giúp mô hình hóa quan hệ user-item phức tạp hơn.

Với implicit feedback, binary cross-entropy/log loss cùng negative sampling là một objective hiệu quả.